SENIORY · GUÍA DE ESTUDIO

Diseño de la base de datos

Los 8 esquemas y 37 tablas de PostgreSQL 17 que sostienen la plataforma: qué guarda cada una, cómo se relacionan y qué reglas hace cumplir la propia base aunque el código falle.

8	37	94	25	95	3	253
esquemas	tablas	reglas CHECK	claves foráneas	índices	triggers	pruebas contra Postgres

Cómo leer los diagramas

PK — Clave primaria: identifica cada fila. Siempre es un UUID v7 generado por la aplicación (salvo catálogos).

FK — Clave foránea real: solo existe dentro del mismo esquema. La línea continua la dibuja.

UQ — Única: no puede repetirse. Los únicos de varias columnas van al pie de la tabla.

ID — Guarda el ID de una tabla de OTRO módulo, sin clave foránea. El texto «→ esquema.tabla» va en el color de ese esquema.

`tipo?` El signo **?** al final del tipo significa que la columna admite **NULL**. Las filas resaltadas en amarillo son las claves del concepto que explica la página.



Línea continua: la «pata de gallo» marca el lado **muchos** (la tabla hija, que tiene la FK) y las dos rayas el lado **uno** (la fila referida). Una raya sola en el lado hijo = relación uno a uno.

Línea punteada (solo en el mapa general): un módulo guarda IDs de otro. No hay clave foránea entre esquemas.

Contenido

2 Mapa general de los 8 esquemas
3 Convenciones del diseño
4 compartido · outbox y auditoría
5 identidad · cuentas, sesiones y consentimientos
6 directorio · cuidadores verificados
7 reservas · servicios y máquina de estados
8 pagos (1/2) · tablas
9 pagos (2/2) · cómo se mueve el dinero
10 teleasistencia · datos de salud
11 notificaciones y mensajeria
12 Cómo la base cumple los escenarios de calidad
13 Conceptos para estudiar
14 Ciclo de vida, roles y decisiones abiertas
15 Autoevaluación

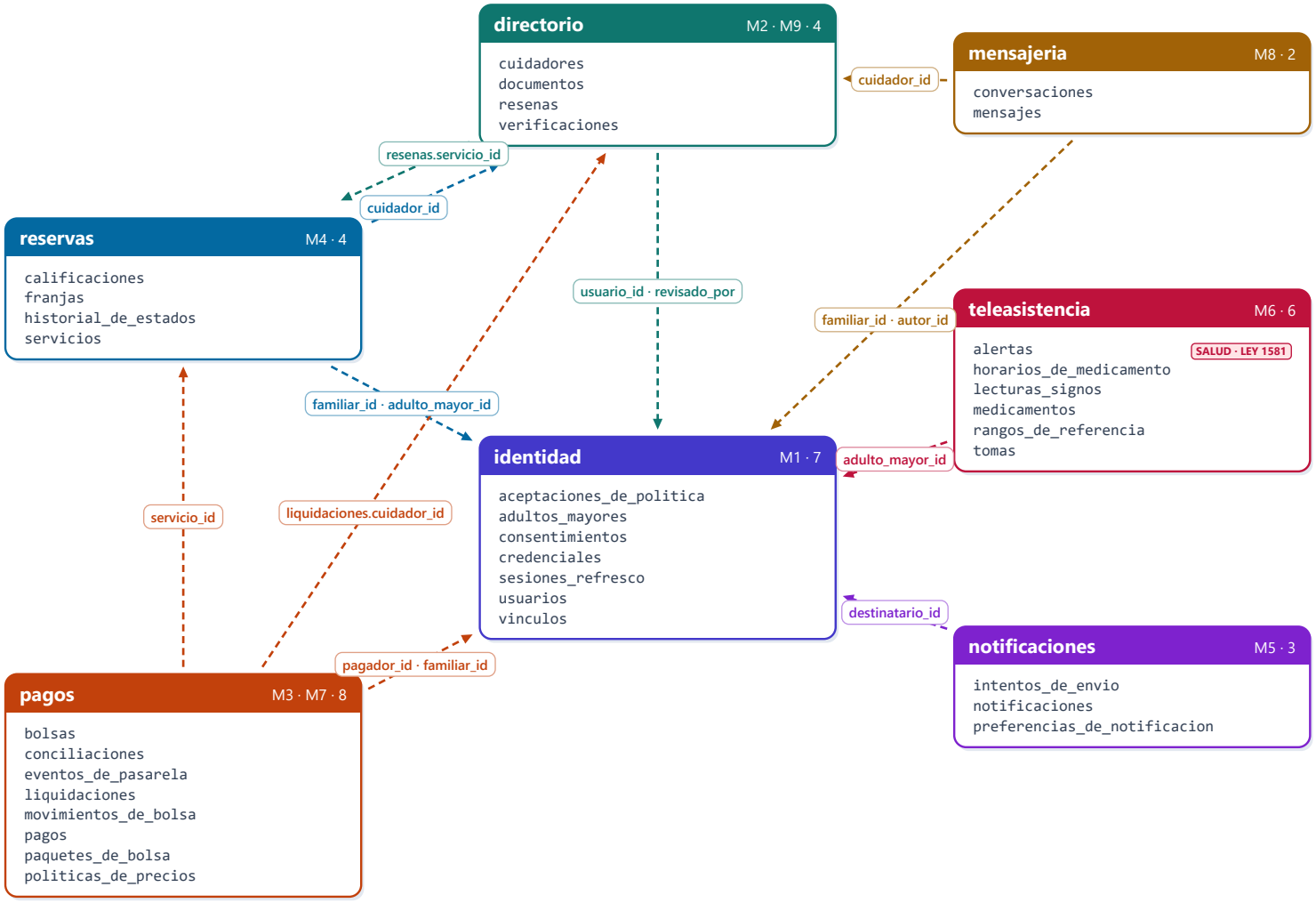
Fuente: la base local construida (información de information_schema y pg_constraint) y BASE_DE_DATOS.md.

26 de octubre de 2026

Mapa general

Un esquema por módulo. Las flechas punteadas son referencias por ID, nunca claves foráneas entre esquemas.

8 esquemas · 37 tablas



compartido · M0 · 3 tablas

`outbox`	`eventos_consumidos`	`auditoria`
eventos por publicar	qué manejador ya procesó cada evento	quién hizo qué (solo inserción)

Todos los módulos escriben aquí dentro de su propia transacción: sus eventos y su auditoría.

IDEA CENTRAL

Cada módulo del monolito es **dueño** de un esquema de Postgres. Ningún módulo lee ni escribe las tablas de otro (escenario **E13** de ADR-01).

CÓMO LEER LAS FLECHAS

A → B significa «A guarda el ID de una fila de B». La etiqueta dice qué columna. No es una clave foránea: la base no comprueba que ese ID exista.

ENTONCES, ¿QUIÉN GARANTIZA LA COHERENCIA?

- Al crear: el caso de uso pregunta al otro módulo por su **puerto** (¿existe el cuidador?, ¿está visible?).
- Al cambiar: el dueño publica un **evento** por el outbox y los demás reaccionan.

QUÉ OBSERVAR

- **identidad** es el centro: casi todos guardan IDs de usuarios o personas mayores.
- **pagos** depende de reservas, identidad y directorio; nadie depende de pagos.
- **teleasistencia** es el único esquema con datos de salud: aislarlo facilita protegerlo y auditarlo.
- **mensajeria** no estaba en el diagrama de arquitectura: hay que añadirlo en el ADR-03.

PARA PENSAR

? *Si mañana Pagos se separara en un microservicio, ¿qué tablas tendría que llevarse? ¿Habría que romper alguna clave foránea?*

Convenciones del diseño

Reglas que se repiten en todas las tablas. Si entiendes esta página, entiendes el 80 % del esquema.

Tema	Regla	Por qué	Ejemplo
Nombres	snake_case, en español; tablas en plural.	El lenguaje del dominio es el del negocio (ADR-01 §4.9).	`adultos_mayores.fecha_de_nacimiento`
Clave primaria	**uuid** generado por la aplicación (UUID v7).	Se conoce antes de insertar y es ordenable por tiempo: los índices se llenan en orden.	`id uuid PK`
Dentro del esquema	Clave foránea real. Por defecto impide borrar el padre si tiene hijos; **ON DELETE CASCADE** solo en credenciales, sesiones, aceptaciones y horarios.	La base impide huérfanos.	`tomas.medicamento_id → medicamentos.id`
Hacia otro esquema	Columna uuid **sin** clave foránea, sufijo _id.	E13: cada módulo es independiente; la coherencia va por puertos y eventos.	`servicios.cuidador_id`
Valores cerrados	**text + CHECK (col IN (...))**, no ENUM.	Añadir un valor es cambiar un CHECK en una migración, no un ALTER TYPE.	`estado IN ('solicitado', ...)`
Dinero	**bigint** en pesos COP enteros, con CHECK ≥ 0.	Sin decimales flotantes: sin errores de redondeo.	`tarifa_hora = 25000`
Porcentajes	**int** en puntos básicos (bps): 8 % = 800.	Aritmética entera exacta.	`comision_familia_bps = 800`
Instantes	**timestamptz** (se guarda en UTC).	Un instante no depende de la zona horaria.	`aceptado_en`
Fechas del negocio	**date** (día civil de America/Bogota) y **time**.	«Hoy» depende de Colombia, no del servidor (hallazgo H2).	`servicios.fecha`
Concurrencia	**version int** en lo que se edita en paralelo.	Bloqueo optimista: si otro cambió la fila, la actualización no aplica.	`servicios · pagos · bolsas · cuidadores · tomas`
Correos	**citext** (texto sin distinguir mayúsculas).	«Ana@x.co» y «ana@x.co» chocan en el UNIQUE.	`usuarios.correo`
Teléfonos	text con CHECK E.164.	Formato internacional único.	`+573001234567`
Búsqueda sin tildes	Columna normalizada + índice GIN trigram.	unaccent() no se puede indexar directamente.	`nombre_busqueda = 'jose perez'`
JSON	jsonb solo para datos sin estructura fija.	Lo que se filtra o valida va en columnas.	`outbox.datos · auditoria.metadatos`
Borrado	No se borran personas: se **anonimizan**.	Ley 1581 (derecho de supresión) sin romper la contabilidad.	`nombre → 'Usuario eliminado'`

Tipos de PostgreSQL que aparecen en los diagramas

`uuid` identificadores (id y referencias)	`text` textos y valores cerrados con CHECK	`citext` texto sin mayúsculas: correo	`bigint` dinero en pesos
`int · smallint` contadores, bps, horas, versiones	`bool` banderas: visible, activo	`date · time` día civil y hora del día	`timestamptz` instante exacto (UTC)
`jsonb` eventos y metadatos sin forma fija	`bytea` bytes: hash del token, sha256	`text[] · smallint[]` listas: disponibilidad, canales, días	`char(3)` moneda: siempre 'COP'

Hallazgo al construirla: índices DESC NULLS FIRST. Drizzle crea los índices descendentes como `DESC NULLS LAST`. Un `ORDER BY creado_en DESC` normal ordena los nulos *primero*, así que Postgres no podía usar ese índice: con 20.000 mensajes recorría toda la tabla y ordenaba en memoria. Como esas columnas son `NOT NULL`, la migración 0009 los recrea como `DESC NULLS FIRST` y el plan pasó a usar el índice (medido con EXPLAIN antes y después).
Lección: un índice solo sirve si su orden coincide exactamente con el de la consulta.

Esquema compartido

Infraestructura común que usan todos los módulos · M0

M0 · 3 tablas · sin datos personales

outbox — 11 col.

PK	id	uuid
	tipo	text
	version	smallint
	datos	jsonb
	id_de_correlacion	uuid
	creado_en	timestamptz
	procesado_en	timestamptz?
	fallido_en	timestamptz?
	intentos	smallint
	proxima_ejecucion	timestamptz
	ultimo_error	text?

índice parcial: (proxima_ejecucion) solo si está pendiente

eventos_consumidos — 3 col.

PK	manejador	text
PK ID	evento_id	→ compartido.outbox
	consumido_en	timestamptz

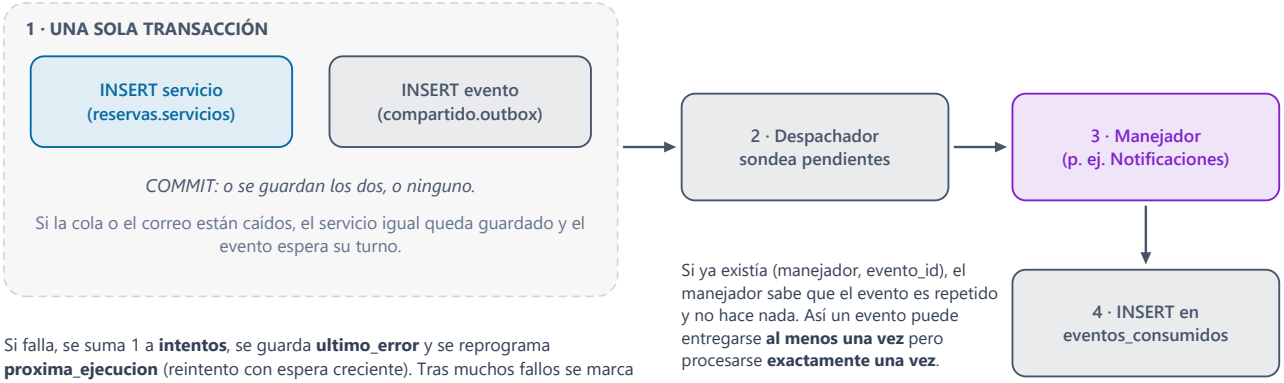
PK compuesta (manejador, evento_id): el mismo evento dos veces choca

auditoria — 9 col.

PK	id	uuid
	ocurrido_en	timestamptz
ID	actor_id	→ identidad.usuarios
	accion	text
	recurso_tipo	text
	recurso_id	text?
	resultado	text
	id_de_correlacion	uuid?
	metadatos	jsonb?

trigger: solo inserción (UPDATE y DELETE fallan)
índice **BRIN** en ocurrido_en · B-tree por recurso y actor

El patrón outbox, paso a paso



Si falla, se suma 1 a **intentos**, se guarda **ultimo_error** y se reprograma **proxima_ejecucion** (reintento con espera creciente). Tras muchos fallos se marca **fallido_en**.

QUÉ GUARDA

Nada del negocio: la «tubería» que conecta los módulos (eventos) y la memoria de quién hizo qué (auditoría).

REGLAS QUE HACE CUMPLIR LA BASE

- **outbox**: el evento se escribe en la misma transacción que el cambio. Sin «doble escritura» (guardar en la base y luego publicar, y que lo segundo falle).
- **eventos_consumidos**: su PK de dos columnas convierte cualquier manejador en **idempotente**.
- **auditoria**: el trigger `auditoria_solo_insercion` rechaza UPDATE y DELETE. Ni un error del código puede borrar el rastro (E6).

ÍNDICES QUE VALE LA PENA ENTENDER

- **Parcial** en outbox: solo indexa lo pendiente. Los millones de eventos ya procesados no estorban al sondeo.
- **BRIN** en auditoria: guarda el mínimo y máximo de cada bloque de filas. Es diminuto y sirve porque las filas llegan en orden de tiempo.

DETALLES

- `datos` y `metadatos` son jsonb, pero nunca llevan datos clínicos ni personales: solo IDs.
- `id_de_correlacion` une todo lo que provocó una misma petición HTTP.

PARA PENSAR

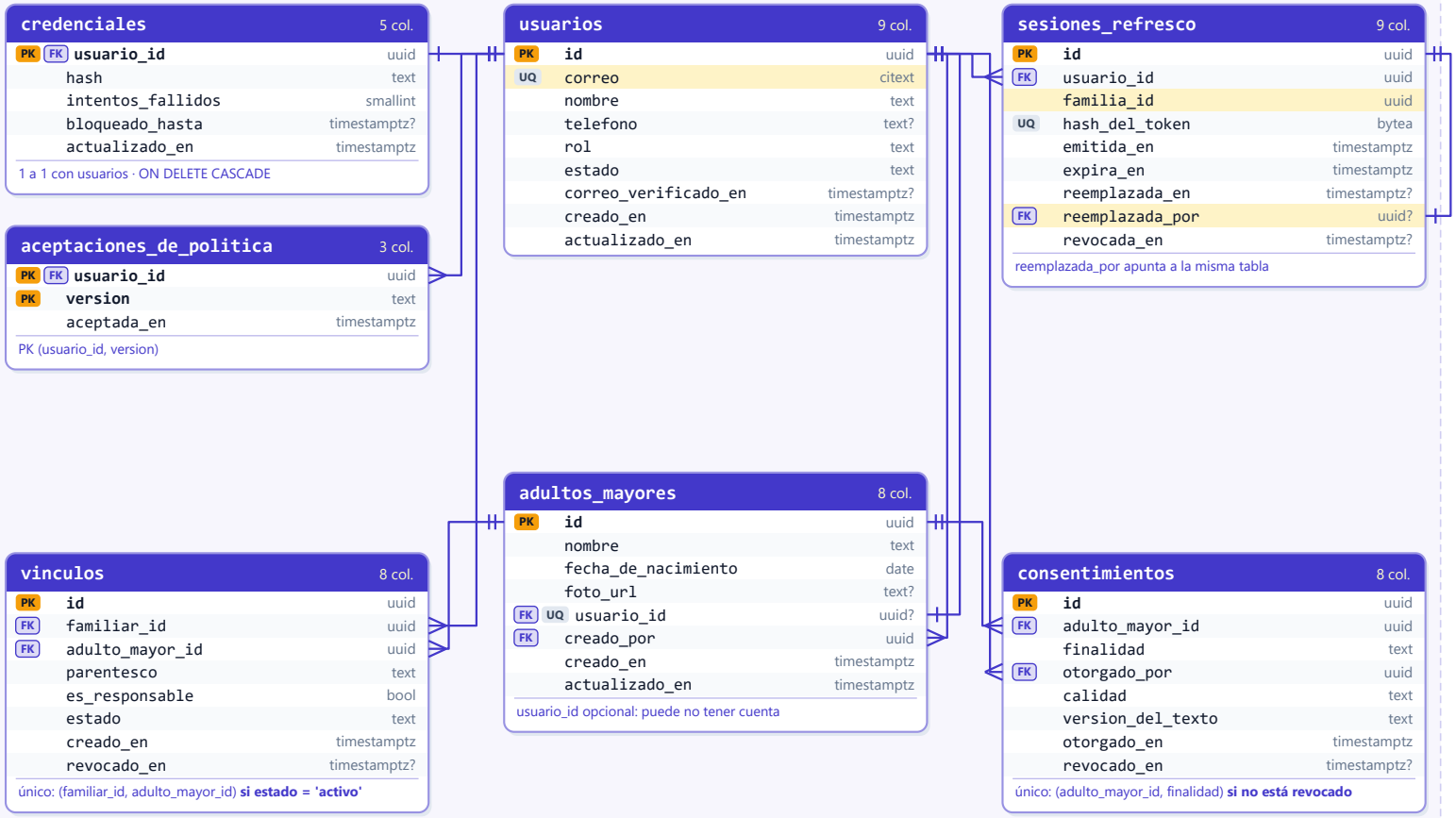
? *¿Qué pasaría si el despachador se cae justo después de entregar el evento pero antes de marcar* `procesado_en`*?*

? *¿Por qué la auditoría guarda IDs y no nombres?*

Esquema identidad

Cuentas, contraseñas, sesiones, personas mayores, vínculos y consentimientos · M1

M1 · 7 tablas · datos personales



Rotación del token de refresco (una «familia»)



Todas comparten **familia_id**. Si alguien presenta el token de A otra vez: dentro de 30 s es una carrera legítima (dos pestañas) y se emite otro par; pasados 30 s es señal de robo y se revoca **toda la familia**.

QUÉ GUARDA

Quién es cada persona, cómo entra y a quién puede ver. **No** guarda datos clínicos: esos viven en teleasistencia.

REGLAS QUE HACE CUMPLIR LA BASE

- **Correo único sin importar mayúsculas** (citext + UNIQUE).
- **credenciales** separada de usuarios: el hash nunca viaja con el perfil y el rol de solo lectura (M10) no la ve.
- Del token de refresco solo se guarda el **hash** (bytea, único). Si roban la base, los tokens no sirven.
- **Únicos parciales**: solo un vínculo activo por pareja y un consentimiento vigente por finalidad, pero los revocados quedan como historial.
- rol IN (familiar, adulto_mayor, cuidador, administrador); teléfono E.164; nombre de 1 a 100 caracteres.

RELACIONES

- usuarios 1—1 credenciales; 1—N sesiones; 1—N vínculos.
- adultos_mayores N—N usuarios a través de **vinculos** (con parentesco y si es responsable).
- **consentimientos** prueban la autorización de la Ley 1581: quién la dio, en qué calidad (titular o representante) y con qué versión del texto.

PARA PENSAR

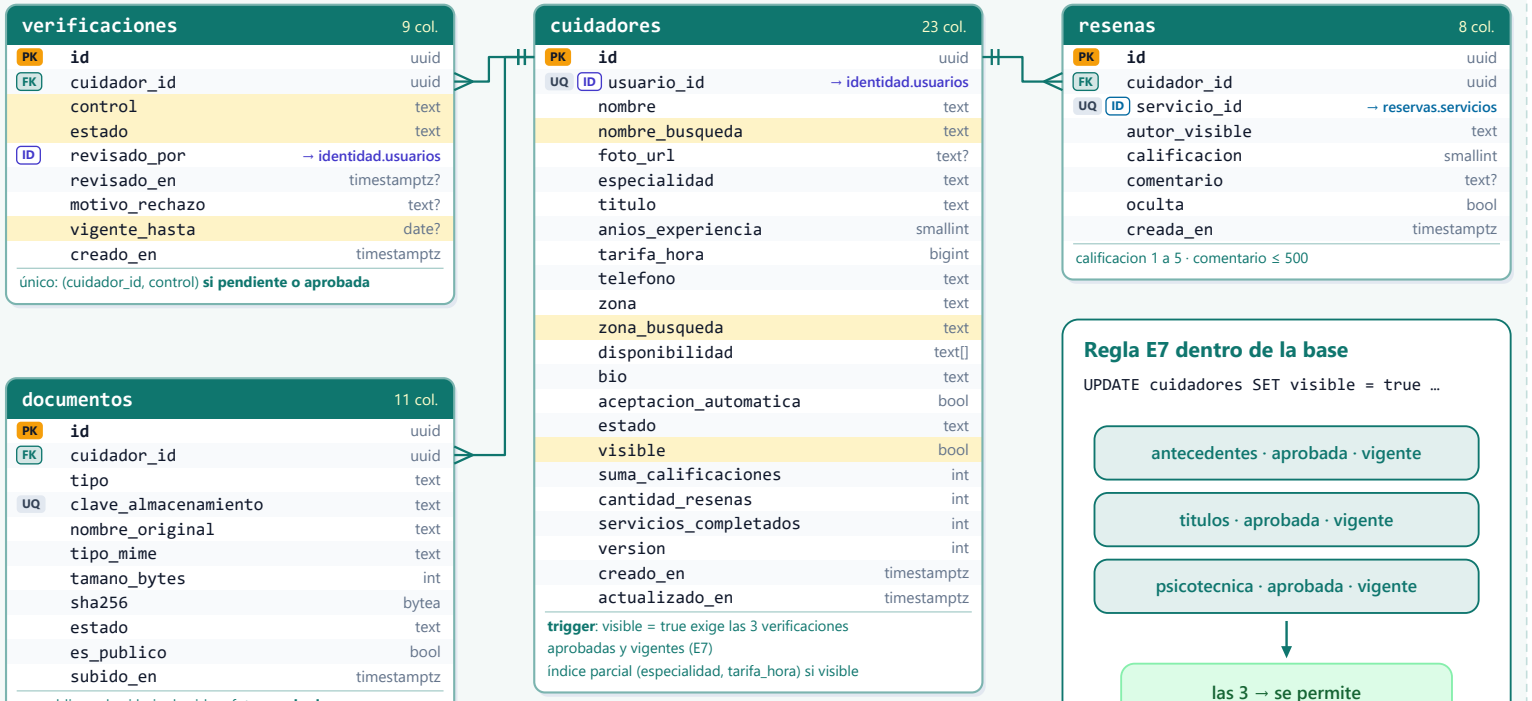
? *¿Por qué no basta un UNIQUE normal en (familiar_id, adulto_mayor_id)?*

? *¿Por qué la edad no se guarda y la fecha de nacimiento sí?*

Esquema directorio

Perfiles de cuidadores, sus verificaciones, documentos y reseñas · M2 (lectura) y M9 (registro)

M2 · M9 · 4 tablas · documentos de identidad



Regla E7 dentro de la base

`UPDATE cuidadores SET visible = true …`

- antecedentes · aprobada · vigente
- titulos · aprobada · vigente
- psicotecnica · aprobada · vigente

↓

las 3 → se permite

Si falta una, el trigger `cuidadores_visible_requiere_verificaciones` lanza un error y la fila no cambia.

Calificación promedio

`suma_calificaciones / cantidad_resenas` se calcula al leer. Ejemplo: 23 / 5 = 4,6. Guardar el promedio acumularía errores de redondeo con cada reseña.

QUÉ GUARDA

El perfil público de cada cuidador y la evidencia de que fue verificado. La familia solo ve cuidadores con **visible = true**.

REGLAS QUE HACE CUMPLIR LA BASE

- **Nadie aparece sin verificar** (E7): trigger sobre visible + única verificación vigente por control.
- Las verificaciones rechazadas o vencidas no se borran: quedan como **historial** (por eso el único es parcial).
- Una **reseña por servicio** (servicio_id UNIQUE): si el evento llega dos veces, no hay dos reseñas.
- disponibilidad solo con manana, tarde o noche, y no vacía; tarifa > 0; experiencia 0 a 60 años.
- Documentos privados por defecto; solo la hoja de vida o la foto aprobadas pueden ser públicas. `sha256` detecta archivos alterados.

BÚSQUEDA

- `nombre_busqueda` y `zona_busqueda` guardan el texto en minúsculas y sin tildes. Un índice **GIN trigram** encuentra «jose» dentro de «José Pérez».
- `disponibilidad` es un arreglo con índice GIN: «¿quién trabaja de noche?» es rápido.

PRIVACIDAD

- El teléfono del cuidador nunca sale en el listado; solo se entrega a quien tiene un servicio.

PARA PENSAR

? *¿Por qué guardar **visible** en vez de calcularla en cada consulta?*

Esquema reservas

El servicio contratado, su historial de estados y su calificación · M4

M4 · 4 tablas · corazón del negocio

franjas — 4 col.

PK	jornada	text
	hora_inicio	time
	hora_fin	time
	cruza_medianoche	bool

datos de referencia: mañana 06–12, tarde 12–18, noche 18–06

historial_de_estados — 7 col.

PK	id	uuid
FK	servicio_id	uuid
	desde	text?
	hacia	text
	accion	text
ID	actor_id	→ identidad.usuarios
	ocurrido_en	timestamptz

una fila por transición (trazabilidad E9)

servicios — 28 col.

PK	id	uuid
ID	familiar_id	→ identidad.usuarios
ID	adulto_mayor_id	→ identidad.adultos_mayores
ID	cuidador_id	→ directorio.cuidadores
	fecha	date
FK	jornada	text
	horas	smallint
	estado	text
	medio_de_pago	text
	aceptacion_automatica	bool
	solicitud_expira_en	timestamptz?
	tarifa_hora	bigint
	subtotal	bigint
	cargo_familia	bigint
	total_familia	bigint
	pago_profesional	bigint
	politica_version	int
	clave_idempotencia	text
	solicitado_en	timestamptz
	aceptado_en	timestamptz?
	iniciado_en	timestamptz?
	finalizado_en	timestamptz?
	cancelado_en	timestamptz?
ID	cancelado_por	→ identidad.usuarios
	motivo_cancelacion	text?
	version	int
	creado_en	timestamptz
	actualizado_en	timestamptz

único: (cuidador_id, fecha, jornada) si estado activo
único: (familiar_id, clave_idempotencia)
CHECK: subtotal = tarifa × horas; total = subtotal + cargo
CHECK: horas 1 a 8 · marcas de tiempo según el estado

calificaciones — 5 col.

PK FK	servicio_id	uuid
ID	familiar_id	→ identidad.usuarios
	calificacion	smallint
	comentario	text?
	creada_en	timestamptz

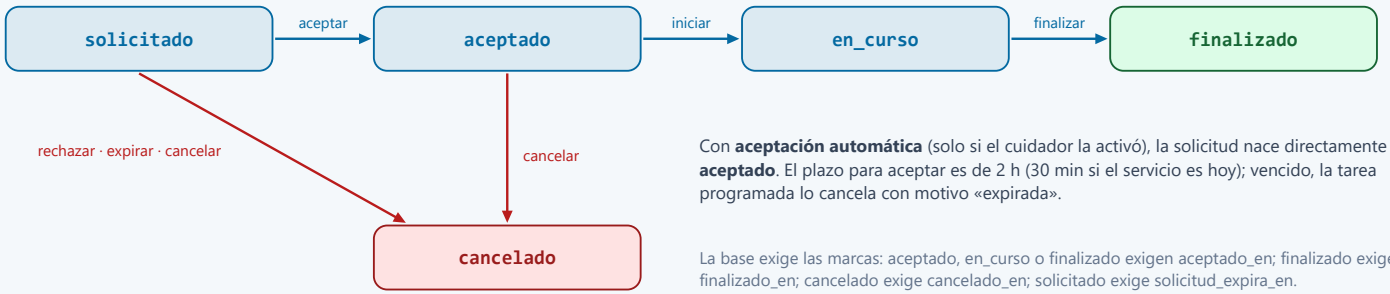
1 a 1 con servicios · calificación 1 a 5

Desglose congelado (ejemplo)

tarifa_hora	$ 25.000
× horas	4
subtotal	$ 100.000
+ cargo_familia (8 %)	$ 8.000
= total_familia	**$ 108.000**
pago_profesional (−10 %)	$ 90.000
ingreso de la plataforma	$ 18.000

Se copia al solicitar con **politica_version**: si mañana cambian la tarifa o las comisiones, este servicio conserva su precio.

Máquina de estados del servicio



Con **aceptación automática** (solo si el cuidador la activó), la solicitud nace directamente en **aceptado**. El plazo para aceptar es de 2 h (30 min si el servicio es hoy); vencido, la tarea programada lo cancela con motivo «expirada».

La base exige las marcas: aceptado, en_curso o finalizado exigen aceptado_en; finalizado exige finalizado_en; cancelado exige cancelado_en; solicitado exige solicitud_expira_en.

QUÉ GUARDA

Cada servicio contratado: quién, para quién, con qué cuidador, qué día y franja, a qué precio y en qué estado va.

LA REGLA ESTRELLA: SIN DOBLE RESERVA

Índice **único parcial** `servicios_sin_doble_reserva` sobre (cuidador_id, fecha, jornada), solo para solicitado, aceptado y en_curso. Un servicio cancelado libera el turno.

Probado: 10 solicitudes simultáneas por el mismo turno → 1 se guarda y 9 reciben violación de unicidad (la API responderá 409).

OTRAS REGLAS

- **Reintentos seguros**: (familiar_id, clave_idempotencia) único. Si el navegador reenvía, no se crea otro servicio.
- **Bloqueo optimista**: `version` evita que aceptar y cancelar a la vez pisen el mismo registro.
- **franjas** es un catálogo con FK: la jornada no puede ser un texto inventado.
- La regla «solo se califica si está finalizado» la aplica el dominio; la base asegura una calificación por servicio.

PARA PENSAR

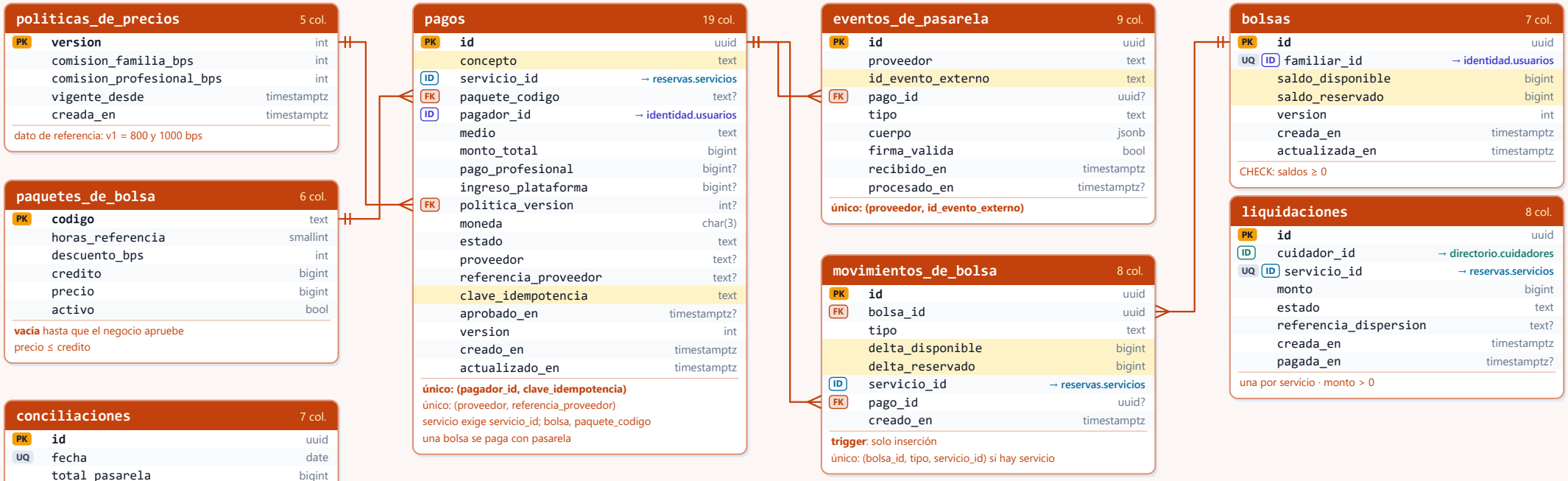
? *¿Por qué el único es parcial? ¿Qué pasaría con un UNIQUE normal cuando un servicio se cancela?*

? *¿Por qué el precio se copia en el servicio en vez de leerse del cuidador?*

Esquema pagos (1/2) · las tablas

Cotización, cobros, avisos de la pasarela, bolsas prepagadas y pagos a cuidadores · M3 y M7

M3 · M7 · 8 tablas · datos financieros



E3 · cero cobros duplicados

- **Clave de idempotencia** por pagador: si el usuario pulsa «Pagar» dos veces, el segundo INSERT choca.
- **Avisos (webhooks)**: la pasarela puede enviar el mismo aviso varias veces; (proveedor, id_evento_externo) lo procesa una sola vez.
- Nunca se guardan datos de tarjeta: solo la referencia del proveedor.

Dinero exacto

- Todo en **bigint** (pesos enteros) y comisiones en **bps**: 800 bps = 8 %.
- `moneda = 'COP'` por CHECK; `monto_total > 0`.
- **pagos.politica_version** registra con qué comisiones se cobró: auditable años después.

Bolsa: un libro contable

- El saldo vive en **bolsas**, pero cada cambio queda como fila en **movimientos_de_bolsa**, que no se edita ni se borra.
- Así la suma de movimientos siempre debe dar el saldo: se puede verificar.
- Siguiente página: los 5 tipos de movimiento con un ejemplo.

Esquema pagos (2/2) · cómo se mueve el dinero

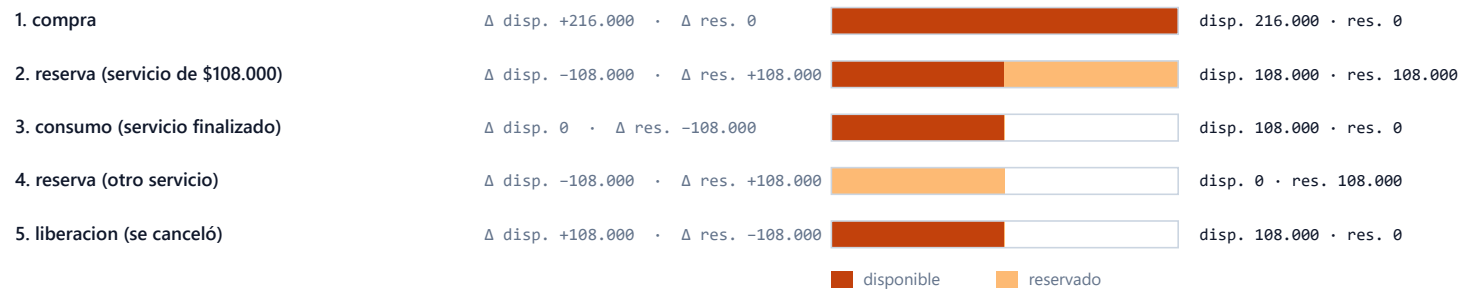
El libro de la bolsa, un ejemplo de saldos y el recorrido de un cobro

unidad de la bolsa: pesos (decisión abierta)

Tipos de movimiento (CHECK movimientos_deltas_coherentes)

tipo	delta_disponible	delta_reservado	cuándo ocurre
`compra`	positivo	0	La familia compra un paquete y la pasarela lo aprueba.
`reserva`	negativo	= −delta_disponible	Contrata un servicio pagando con bolsa: el dinero se aparta.
`consumo`	0	negativo	El servicio finaliza: lo apartado se gasta.
`liberacion`	positivo	= −delta_disponible	El servicio se cancela: lo apartado vuelve.
`ajuste`	cualquiera	0	Corrección manual de un administrador (queda auditada).

Ejemplo: saldos de una bolsa paso a paso



Cada paso es **una transacción**: se bloquea la fila de la bolsa (`SELECT … FOR UPDATE`), se inserta el movimiento y se actualizan los saldos. Si un saldo quedara negativo, el CHECK `bolsas_saldos_no_negativos` aborta todo. El único parcial (bolsa_id, tipo, servicio_id) impide reservar o consumir dos veces por el mismo servicio. Otros dos CHECK exigen que una compra tenga `pago_id` y que reserva, consumo y liberación tengan `servicio_id`.

Fórmulas de la cotización (política v1, aritmética entera)

subtotal	= tarifa_hora × horas	25.000 × 4 = 100.000
cargo_familia	= subtotal × comision_familia_bps / 10.000	100.000 × 800 / 10.000 = 8.000
total_familia	= subtotal + cargo_familia	100.000 + 8.000 = 108.000
pago_profesional	= subtotal − subtotal × comision_profesional_bps / 10.000	100.000 − 10.000 = 90.000
ingreso_plataforma	= total_familia − pago_profesional	108.000 − 90.000 = 18.000

Las tres primeras líneas las vigila un CHECK en **reservas.servicios** y la coherencia del desglose otro en **pagos.pagos**.

Recorrido de un cobro con pasarela

1. **Cotizar (M3)**
   Se lee la política vigente y se calcula el desglose. No se guarda nada.
2. **Cuidador acepta**
   Decisión del 2026-10-26: se cobra al aceptar.
3. **INSERT pagos**
   estado = pendiente, con la clave de idempotencia.
4. **Pasarela**
   La familia paga en la página del proveedor.
5. **Webhook**
   INSERT eventos_de_pasarela (único por id externo); se verifica la firma.
6. **pagos → aprobado**
   Se publica PagoAprobado por el outbox.
7. **Finaliza el servicio**
   INSERT liquidaciones (una por servicio) con el pago al cuidador.
8. **Conciliación diaria**
   Compara totales de la pasarela con los internos (una fila por fecha).

Para pensar: *¿por qué se guarda el aviso de la pasarela antes de procesarlo, y qué pasa si llega dos veces?*

Esquema teleasistencia

Medicamentos, tomas, signos vitales, rangos de referencia y alertas · M6

M6 · 6 tablas · DATOS DE SALUD (Ley 1581)

medicamentos — 12 col.

PK	id	uuid
ID	adulto_mayor_id	→ identidad.adultos_mayores
	nombre	text
	dosis	text
	indicaciones	text?
	dias_semana	smallint[]
	vigente_desde	date
	vigente_hasta	date?
	activo	bool
ID	creado_por	→ identidad.usuarios
	creado_en	timestamptz
	actualizado_en	timestamptz

dias_semana entre 1 y 7 (1 = lunes), no vacío
vigente_hasta ≥ vigente_desde

horarios_de_medicamento — 3 col.

PK	id	uuid
FK	medicamento_id	uuid
	hora	time

único: (medicamento_id, hora) · CASCADE

tomas — 10 col.

PK	id	uuid
FK	medicamento_id	uuid
ID	adulto_mayor_id	→ identidad.adultos_mayores
	fecha	date
	hora_programada	time
	estado	text
ID	registrada_por	→ identidad.usuarios
	registrada_en	timestamptz?
	version	int
	creada_en	timestamptz

único: (medicamento_id, fecha, hora_programada)
tomada exige registrada_en

rangos_de_referencia — 11 col.

PK ID	adulto_mayor_id	→ identidad.adultos_mayores
	sistolica_min	smallint?
	sistolica_max	smallint?
	diastolica_min	smallint?
	diastolica_max	smallint?
	frecuencia_min	smallint?
	frecuencia_max	smallint?
	glucosa_min	smallint?
	glucosa_max	smallint?
ID	definidos_por	→ identidad.usuarios
	actualizados_en	timestamptz

1 a 1 con la persona; sin fila = valores por defecto
mínimo < máximo en cada par

Cómo nacen las tomas del día



Si la tarea corre dos veces, el único choca y no se duplican. Luego cada toma pasa a **tomada** u **omitida**.

lecturas_signos — 11 col.

PK	id	uuid
ID	adulto_mayor_id	→ identidad.adultos_mayores
	tomada_en	timestamptz
	presion_sistolica	smallint?
	presion_diastolica	smallint?
	frecuencia_cardiaca	smallint?
	glucosa	smallint?
	fuera_de_rango	bool
	fuente	text
ID	registrada_por	→ identidad.usuarios
	creada_en	timestamptz

al menos una medida · diastólica < sistólica
rangos plausibles (p. ej. glucosa 20–600)

alertas — 11 col.

PK	id	uuid
ID	adulto_mayor_id	→ identidad.adultos_mayores
	origen	text
	estado	text
ID	generada_por	→ identidad.usuarios
	clave_idempotencia	text
	generada_en	timestamptz
	primera_entrega_en	timestamptz?
ID	atendida_por	→ identidad.usuarios
	atendida_en	timestamptz?
	nota	text?

único: (generada_por, clave_idempotencia)
E11: primera_entrega_en − generada_en < 30 s

POR QUÉ ES ESPECIAL

Es el **único** esquema con datos de salud, que la Ley 1581 trata como **sensibles**. Concentrarlos aquí permite protegerlos con reglas propias.

- Toda lectura (y toda negación) queda en `compartido.auditoria` (E6).
- Solo se accede con vínculo activo y consentimiento vigente (identidad).
- Candidatas a **cifrado de columnas**: nombre e indicaciones del medicamento (revelan diagnósticos) y la nota de la alerta. *Decisión abierta.*

REGLAS QUE HACE CUMPLIR LA BASE

- Generar las tomas es **idempotente**: (medicamento_id, fecha, hora_programada) único.
- Signos: puede faltar una medida, pero no todas; sistólica y diastólica van juntas; valores imposibles se rechazan.
- **Alerta sin duplicados**: un reintento por mala conexión no crea otra alerta. Probado con dos pulsaciones simultáneas: solo una gana.

DESNORMALIZACIÓN A PROPÓSITO

`tomas.adulto_mayor_id` repite un dato que ya está en medicamentos. Así «las tomas de hoy de doña Carmen» se responde con un índice, sin JOIN.

PARA PENSAR

? *¿Quién debe definir los rangos de referencia y por qué no los programadores?*

? *¿Cómo medirías desde SQL el percentil 95 de E11?*

Esquemas notificaciones y mensajeria

Avisos a las personas por cada canal · M5 · Chat entre familias y cuidadores · M8

M5 · 3 tablas · M8 · 2 tablas

notificaciones · M5

notificaciones — 14 col.

PK	id	uuid
ID	destinatario_id	→ identidad.usuarios
ID	evento_id	→ compartido.outbox
	tipo	text
	prioridad	text
	canal	text
	estado	text
	parametros	jsonb
	intentos	smallint
	proximo_intento_en	timestamptz
	creada_en	timestamptz
	enviada_en	timestamptz?
	entregada_en	timestamptz?
	ultimo_error	text?

único: (evento_id, destinatario_id, tipo, canal)
índice parcial: pendientes por proximo_intento_en

intentos_de_envio — 5 col.

PK	id	uuid
FK	notificacion_id	uuid
	intentado_en	timestamptz
	resultado	text
	respuesta_proveedor	text?

resultado: exito o error

preferencias_de_notificacion — 4 col.

PK ID	usuario_id	→ identidad.usuarios
	canales	text[]
	silencio_desde	time?
	silencio_hasta	time?

silencio: desde y hasta, o ninguno
las alertas ignoran el silencio

mensajeria · M8

conversaciones — 5 col.

PK	id	uuid
ID	familiar_id	→ identidad.usuarios
ID	cuidador_id	→ directorio.cuidadores
	creada_en	timestamptz
	ultimo_mensaje_en	timestamptz?

único: (familiar_id, cuidador_id)

mensajes — 8 col.

PK	id	uuid
FK	conversacion_id	uuid
ID	autor_id	→ identidad.usuarios
	autor_rol	text
	texto	text
	id_de_cliente	uuid
	creado_en	timestamptz
	leido_en	timestamptz?

único: (conversacion_id, autor_id, id_de_cliente) · texto 1 a 1000
índice (conversacion_id, creado_en DESC, id)

NOTIFICACIONES

- Nace de un **evento** (ServicioConfirmado, AlertaGenerada…). Su único de 4 columnas hace que reprocesar el evento no repita el aviso, pero sí permite avisar por otro canal o a otra persona.
- **Cola de reintentos** en la propia tabla: estado, intentos y proximo_intento_en, con índice parcial solo de las pendientes.
- `parametros` (jsonb) nunca lleva datos clínicos: un SMS puede leerlo cualquiera que vea el teléfono.
- Canales: app, correo, sms, whatsapp, push. Prioridad normal o alta.

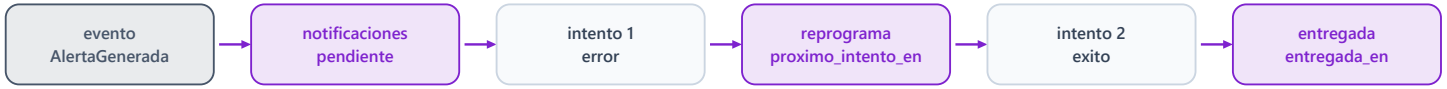
MENSAJERIA

- **Obtener o crear**: una sola conversación por pareja familia–cuidador; si dos peticiones llegan a la vez, el único decide.
- `id_de_cliente` lo genera el navegador. Si la conexión falla y reenvía, el mensaje no se duplica.
- Solo escriben el familiar o el cuidador (autor_rol).

PARA PENSAR

- ? *¿Por qué las alertas de teleasistencia deben ignorar el horario silencioso?*
- ? *¿Qué ventaja tiene el cursor sobre OFFSET cuando llegan mensajes nuevos?*

Vida de una notificación



Cada intento queda en **intentos_de_envio** (con la respuesta del proveedor). Para E11 interesa la primera entrega de una alerta.

Paginación por cursor (mensajes)

```
SELECT … WHERE conversacion_id = $1 AND (creado_en, id) < ($cursor_fecha, $cursor_id) ORDER BY creado_en DESC, id DESC LIMIT 30
```

En vez de «página 7» (OFFSET, que se vuelve lento y salta mensajes cuando llegan nuevos), se pide «los 30 anteriores a este». El índice tiene exactamente ese orden, así que Postgres lee solo 30 entradas. Se agrega **id** como desempate porque dos mensajes pueden tener el mismo instante. Aquí se descubrió el problema de **NULLS LAST** (página 3).

Cómo la base cumple los escenarios de calidad

Las reglas críticas no dependen solo de que el código esté bien: la base las respalda.

Escenario (ADR-01)	Riesgo	Mecanismo en la base	Dónde
E3 · 0 cobros duplicados	Doble clic, reintentos, webhooks repetidos.	UNIQUE (pagador_id, clave_idempotencia); UNIQUE (proveedor, id_evento_externo); saldos ≥ 0; un movimiento por servicio y tipo.	pagos
E6 · Datos clínicos protegidos	Lecturas sin permiso, rastro borrado.	Datos de salud solo en teleasistencia; auditoría de solo inserción (trigger); en M10, roles por módulo.	teleasistencia · compartido
E7 · Solo cuidadores verificados	Un cuidador sin antecedentes aparece en el directorio.	Trigger cuidadores_visible_requiere_verificaciones; una verificación vigente por control; índice parcial solo de visibles.	directorio
E9 · Estados correctos	Un servicio finalizado sin haberse aceptado.	CHECK de estados y de marcas de tiempo; historial_de_estados; version (bloqueo optimista).	reservas
E10 · Costo correcto	Totales que no cuadran.	Desglose congelado con CHECK aritmético; dinero en bigint; comisiones en bps con versión de política.	reservas · pagos
E11 · Alerta en < 30 s	Alertas duplicadas o lentas sin medición.	UNIQUE (generada_por, clave_idempotencia); generada_en y primera_entrega_en para medir el p95.	teleasistencia
E13 · Módulos independientes	Un módulo lee o bloquea tablas de otro.	Un esquema por módulo y **0 claves foráneas entre esquemas** (lo verifica una prueba); en M10, permisos por esquema.	todos
Doble reserva	Dos familias contratan el mismo turno.	Único parcial (cuidador_id, fecha, jornada) en estados activos. Probado con 10 solicitudes simultáneas.	reservas

Coherencia entre módulos sin claves foráneas

Situación	Cómo se resuelve
Crear algo que apunta a otro módulo	El caso de uso consulta el **puerto de entrada** del otro módulo antes de guardar (¿existe?, ¿está visible?).
El otro módulo cambia	Publica un **evento** y los interesados reaccionan (p. ej. un cuidador suspendido → Reservas revisa sus servicios futuros).
Se elimina una cuenta	Evento cuenta_eliminada → cada módulo **anonimiza** sus datos de esa persona.
Un ID quedó huérfano por un error	Consulta de verificación periódica (opcional, M10) que lista IDs sin dueño.

Defensa en profundidad

API (M0 · M1)
valida formato y permisos: class-validator, guardias

Dominio (cada módulo)
reglas de negocio: «solo se califica si está finalizado»

Base de datos
CHECK, UNIQUE, FK y triggers: la última barrera

Las pruebas que lo demuestran

- **253 pruebas de integración** contra un PostgreSQL 17 real (Testcontainers), no contra una imitación.
- Las pruebas intentan violar las reglas a propósito y comprueban el **nombre exacto** de la restricción que lo impide.
- **Concurrencia real**: 10 reservas simultáneas del mismo turno → 1 gana; 2 alertas simultáneas → 1 gana.
- Las pruebas de estructura confirman **0 claves foráneas entre esquemas**, las 37 tablas, PK en todas, dinero en enteros y timestamptz.
- Los triggers se prueban intentando UPDATE y DELETE sobre auditoría y el libro de la bolsa.

Conceptos para estudiar

Las ideas de bases de datos que aparecen en este diseño, explicadas con un ejemplo de Seniory.

Clave primaria y UUID v7

Identifica cada fila. El UUID v7 empieza por la hora, así que los nuevos quedan al final del índice (como un autoincremental) pero se generan en la aplicación sin consultar la base.

Clave foránea (FK)

Obliga a que el valor exista en otra tabla. Aquí solo dentro de un esquema: `tomas.medicamento_id` no puede apuntar a un medicamento inexistente.

Referencia por ID sin FK

Entre módulos se guarda el ID pero la base no lo verifica. Se gana independencia (E13) y se paga con coherencia «eventual» mediante eventos.

CHECK

Una condición que toda fila debe cumplir. Ej.: `horas BETWEEN 1 AND 8`. Hay 94 en el esquema; son la primera línea de defensa.

Único parcial

UNIQUE solo para las filas que cumplen un WHERE. Permite historial: muchos vínculos revocados, pero solo uno activo por pareja.

Idempotencia

Repetir una operación da el mismo resultado que hacerla una vez. Se logra con una clave única (clave_idempotencia, id_de_cliente, id_evento_externo).

Bloqueo optimista

Columna `version`: UPDATE … WHERE id = $1 AND version = $2. Si otro cambió la fila antes, afecta 0 filas y se reintenta o se informa.

Transacción

Un grupo de cambios que se aplica completo o no se aplica. El outbox y el libro de la bolsa dependen de esto.

Patrón outbox

El evento se guarda en la misma transacción que el cambio; otro proceso lo publica después. Evita perder eventos o publicar cambios que no se guardaron.

Trigger

Código que la base ejecuta ante un INSERT/UPDATE/DELETE. Aquí: auditoría y libro de bolsa de solo inserción, y la regla E7 de visibilidad.

Índice B-tree

El índice normal: ordena valores para buscar y ordenar rápido. Su orden (ASC/DESC, NULLS FIRST/LAST) debe coincidir con la consulta.

Índice parcial

Solo indexa filas que cumplen una condición: outbox pendiente, cuidadores visibles, notificaciones pendientes. Más pequeño y rápido.

GIN y pg_trgm

GIN indexa partes de un valor (elementos de un arreglo o trigramas de un texto). Con pg_trgm, «LIKE '%jose%'» usa índice.

BRIN

Guarda el rango de valores de cada bloque de la tabla. Diminuto; ideal para columnas que crecen en orden (ocurrido_en en auditoría).

Normalización y desnormalización

Normalizar: cada dato en un solo lugar. A veces se duplica a propósito para leer rápido (tomas.adulto_mayor_id) o para congelar (precio del servicio).

timestamptz vs date

timestamptz es un instante (UTC). date es un día del calendario de Bogotá. «El servicio es el martes» es date; «se aceptó a las 3:02 p. m.» es timestamptz.

Migraciones

Archivos SQL numerados (0000…0009) que llevan la base de una versión a otra. Antes de producción se pueden aplanar; después nunca se editan.

Datos de referencia vs. de prueba

Franjas y política de precios van a producción dentro de migraciones. Los datos de ejemplo (fixtures) nunca salen de desarrollo.

Ciclo de vida, roles y decisiones abiertas

Qué pasa con los datos con el tiempo, quién puede tocarlos y qué falta decidir.

Ciclo de vida de los datos (Ley 1581)

Dato	Retención	Al eliminar la cuenta
usuarios	Mientras exista la cuenta	nombre → «Usuario eliminado», correo → eliminado+<id>@invalido, teléfono nulo, estado eliminado
sesiones_refresco	30 días tras vencer	Se revocan y se borran
adultos_mayores y teleasistencia	**Abierta** (asesoría legal)	Si nadie queda vinculado: anonimizar y borrar los datos de salud
consentimientos	Se conservan como prueba	Se marcan revocados; no se borran
servicios, pagos, liquidaciones	Obligaciones contables **(abierta)**	Se conservan con IDs; el nombre ya está anonimizado
mensajes	**Abierta** (propuesta 12 meses)	Texto → «Mensaje eliminado»
resenas	Mientras el cuidador esté activo	autor_visible → «Familia de Seniory»
auditoria	**Abierta** (propuesta 2 años)	Se conserva (solo IDs)

Roles de base de datos (M10)

Rol	Permisos
`seniory_migraciones`	Dueño de los esquemas; solo lo usa la CI al migrar.
`seniory_<modulo>`	Lee y escribe **solo su esquema**; inserta en outbox, eventos_consumidos y auditoría.
`seniory_lectura`	Solo lectura para soporte; **sin** teleasistencia ni credenciales.

Con estos roles, aunque el código tuviera un error, un módulo **no podría** leer las tablas de otro: E13 pasa de ser una convención a una garantía de la base.

Decisiones abiertas que afectan la base

#	Decisión	Tablas afectadas	Recomendación	Quién decide
1	Unidad de la bolsa: pesos u horas	bolsas, movimientos_de_bolsa, paquetes_de_bolsa	Pesos (las tarifas varían de $18.000 a $35.000 por hora)	Equipo empresarial
2	Paquetes y descuentos de la bolsa	paquetes_de_bolsa (hoy vacía)	Aprobar antes de M7; los de la landing son ilustrativos	Equipo empresarial
3	Rangos de referencia de signos vitales	rangos_de_referencia	Definidos por un profesional de salud	Profesional de salud
4	Cifrado de columnas clínicas	medicamentos, alertas	Sí para nombre, indicaciones y nota; elegir pgcrypto o cifrado en la aplicación en M6	Equipo técnico
5	Plazos de retención	varias (tabla de arriba)	Con asesoría legal antes de producción	Asesoría legal
6	Catálogo de zonas	cuidadores	Texto libre hasta que el directorio crezca	Equipo técnico
OK	Horario de las franjas	franjas	**Resuelto (2026-10-26):** 06–12, 12–18, 18–06	—

Migraciones construidas (backend/drizzle/)

`0000` extensiones · `0001` compartido · `0002` identidad · `0003` directorio · `0004` pagos · `0005` reservas · `0006` notificaciones · `0007` teleasistencia · `0008` mensajeria · `0009` índices DESC

Una migración por módulo, en orden de dependencias. Antes del primer despliegue a producción se pueden aplanar; desde ese momento **nunca se editan**: cada cambio es una migración nueva.

Autoevaluación

Responde antes de mirar la respuesta (en gris). Si fallas una, vuelve a la página indicada.

1 ¿Por qué no hay claves foráneas entre esquemas?

Para que cada módulo sea independiente (E13): se puede cambiar, migrar o separar sin romper a los demás. La coherencia va por puertos y eventos. (p. 2)

2 ¿Qué impide que dos familias reserven al mismo cuidador en el mismo turno?

El índice único parcial (cuidador_id, fecha, jornada) solo en solicitado, aceptado y en_curso. (p. 7)

3 ¿Por qué ese único es parcial?

Porque un servicio cancelado o finalizado debe liberar el turno; un UNIQUE normal bloquearía la fecha para siempre. (p. 7)

4 Si el usuario pulsa «Pagar» dos veces, ¿por qué no se cobra dos veces?

Ambas peticiones llevan la misma clave de idempotencia y UNIQUE (pagador_id, clave_idempotencia) rechaza la segunda. (p. 8)

5 ¿Por qué el dinero es bigint y no decimal o float?

Los pesos no tienen centavos en la práctica y los enteros no tienen errores de redondeo; las comisiones van en bps. (p. 3)

6 ¿Cómo se sabe el saldo de una bolsa si se duda de él?

Sumando sus movimientos, que no se pueden editar ni borrar (trigger de solo inserción). (p. 9)

7 ¿Qué pasa si el código intenta marcar visible a un cuidador sin psicotécnica?

El trigger cuidadores_visible_requiere_verificaciones lanza un error y la fila no cambia (E7). (p. 6)

8 ¿Para qué sirve eventos_consumidos?

Para que cada manejador procese un evento exactamente una vez aunque se entregue varias veces: PK (manejador, evento_id). (p. 4)

9 ¿Por qué el precio se copia en el servicio?

Para congelarlo: si cambian la tarifa o las comisiones, el servicio conserva el precio pactado (E10). (p. 7)

10 ¿Qué diferencia hay entre timestamptz y date aquí?

timestamptz es un instante en UTC (aceptado_en); date es un día del calendario de Bogotá (fecha del servicio). (p. 13)

11 ¿Por qué un índice DESC NULLS LAST no servía?

La consulta ordenaba DESC, que pone los nulos primero; el orden del índice no coincidía y Postgres recorría la tabla. (p. 3)

12 ¿Dónde viven los datos de salud y qué protección tienen?

Solo en teleasistencia; acceso con vínculo y consentimiento, auditoría de cada lectura y cifrado de columnas por decidir. (p. 10)

13 ¿Por qué paquetes_de_bolsa está vacía?

Porque los paquetes y la unidad (pesos u horas) no están aprobados; poner precios sin aprobación sería prometer algo falso. (p. 8)

14 ¿Qué es una clave de idempotencia en una alerta?

Un valor que manda el dispositivo; si reintenta por mala conexión, UNIQUE (generada_por, clave) evita una segunda alerta. (p. 10)